

Seasons in Teamder

A complete technical reading of the seasons feature, written for a reviewer who has never seen this codebase.

Repository `~/Projects/soccer` , branch `perf/firestore-read-costs` . Every line number is against commit `e35394a` (2026-09-18 18:40 UTC), the head of that branch. Every production figure was read live from Firestore project `soccer-app-52b6b` on **2026-09-19**.

Orientation — what a season is, and why the feature exists

Teamder is a Hebrew, right-to-left app for organising amateur football. Its unit of social organisation is a **club** (`groups/{groupId}`): a group of regulars who play together, typically once a week. Its unit of activity is an **evening** (`games/{gameId}`): one game-night, with a roster, arrivals, and — when it is over — a per-player statistical footprint. The database holds 195 clubs, 675 accounts and 95 game documents.

Before seasons, every number the app showed about a club was cumulative for the life of the club. The goal-scoring table, the clean-sheet percentages, the club's level and badges, the chemistry between pairs of players: all of it accumulated forever, in three running rollup collections (`communityPlayerStats` , `communityPairStats` , `communityStats`) that nothing ever reset. A member who joined in year three could not catch a member who joined in year one, and a club had no occasion — no ceremony, no closing of a book — on which to say *that was the year, and here is who won it*.

A **season** is the window the feature introduces. Concretely, in this codebase, a season is:

- a small mutable record on the club document, `groups/{groupId}.seasons` , carrying the season's number, its id (`s1` , `s2` , ...), when it started, how far it has got, and where its finish line is;
- a **finish line** — either a number of evenings (the *rounds* cadence) or a calendar date (the *date* cadence);
- and, once the finish line is reached, an irreversible ceremony: the club's three running rollup tables are copied into one write-once archive document, nine titles are decided and written onto the winners' profiles, every participant is pushed a personal summary, and the live tables are wound back to zero so the next season starts empty.

The pitch to the admin is in the app's own words (`he.seasonsToggleInfo`): the club gets a fresh start, a hall of fame, and an end-of-season ceremony. One promise in particular shapes almost everything in this document: when an existing club switches the feature on, **its entire history becomes season 1**. That promise is implemented not by writing anything onto the old evenings but by a rule about *absence* — an evening with no season stamp belongs to season 1 — and that rule is the root of a large fraction of the defects below.

The life of a season, in the order the code lives it

#	What happens	Written by	Section
1	An admin turns the feature on, picks a cadence and a target, and chooses whether the club's history is <i>carried into</i> season 1 or <i>sealed as</i> season 1	<code>enableClubSeasons</code>	§2

#	What happens	Written by	Section
2	Each evening is played; when it finishes it is sealed , which credits the players, writes an evening summary, and moves the season's progress counter by one	onGameRosterChanged → sealRoundSummary	§3
3	Each evening is stamped with the running season's id on its way into active or finished	onGameRosterChanged	§3
4	Two mechanisms watch for the finish line: the seal itself (rounds cadence only) and an hourly sweep (all cadences)	closeSeasonIfRoundsTargetMet , runSeasonRollovers	§4
5	The close : archive, card, nine titles, push, wind the live tables back, open the next season	closeSeason + performSeasonClose	§5
6	Optionally, within a grace period, an admin presses undo	reopenLastSeason + reopenSeason	§6
7	The sealed season is read for ever after by the hall of fame, the statistics screen's season picker, the personal summary and the profile title shelf	client only	§10–§11

Two words the reader must not confuse

Hebrew	Transliteration	What it is	Which clubs have it
מחזור	<i>machzor</i>	one game-night — a whole evening. The unit a season's length is measured in.	every club
משחקון	<i>mischakon</i>	one mini-game inside that night , typically four to six per evening	only clubs using "advanced mode"

משחקונים are recorded only when an admin runs the evening from the *advanced live screen*, which commits each mini-game separately (`commitRoundStats`). The common club runs a plain timer and records **zero** משחקונים for every season it will ever play. Almost every one of the nine season titles, and most of the tiles on the personal summary, are computed from mini-game counters. A great many defects in this document are one mistake in different clothes: a screen reads one of these two units and labels it with the other, or divides one by the other. Where a field in the code is called `rounds` and means mini-games, this document says so at the point of use.

The population this feature runs on

This matters more than usual, because the feature is almost entirely untried:

- **195** `groups` documents in production (paged count, 19.09.2026).
- **2** of them carry a `seasons` block at all. **Both** use the rounds cadence. **No club has ever run a date season** — every statement about date cadence in §4 is read from code and verified by arithmetic, never observed.
- **1** club — `HhzIwmjM1i5HS0GHt3p` , מועדון שכחת שושי ("Shoshi's Forgetting Club"), seven members — has ever closed a season. It is the worked example throughout, because there is no other. There is exactly **one** `seasonSummary` document and **one** `seasonCards` document in the entire database.
- The one club that has run a season is an *advanced-mode* club, so every rules clause, every title and every screen that depends on `rounds > 0` has only ever been exercised on the uncommon case.

Code paths that only fire on a club's second, third or fourth season have never executed against real data. That is why so many of the defects below survived to production.

The three counters, named once here

"How many מחזורים has this club played" has three independently-written answers, and they are allowed to disagree. §3 develops this; the short version:

Counter	Where	Who writes it	Resets?	Value on the one real club today
clubRecords.eveningsSealed	/clubRecords/{groupId}	server, increment(1) per sealed evening, since 26.08.2026	never	10
seasons.playedRounds	groups.seasons map	server: seeded at open, increment(1) per seal	at every season opening	0
a live scan of games	the phone	nobody — derived on every screen open	n/a	22

The club has played 22 evenings. Each of those three numbers is defensible on its own terms and they are the origin of the complaint that produced this document: one screen says 22, the next says 0.

Provenance: what was read, what is running, and what is still only in git

How this was produced

Every claim here was made by reading two things: the source at commit `e35394a`, and the production database on 2026-09-19. Production was read over the Firestore REST API with an owner access token (`gcloud auth print-access-token`), using `documents:runQuery` with an explicit `orderBy` — plain `documents.list` reads on this project return stale results. **Nothing was written, no callable was invoked and nothing was deployed** in the course of writing this.

Two practical notes for a reviewer who wants to re-run any of it:

- Line numbers are against the **commit**, not the working tree. The tree currently carries an unrelated uncommitted change (a display-name/email validation rule) that touches `firestore.rules` and adds two lines near the top of `src/i18n/he.ts`; check out `e35394a` before matching line numbers.
- The Firebase Rules API returns `403` for these credentials unless a quota project is supplied. Add `-H "x-goog-user-project: soccer-app-52b6b"` and it answers.

What is running where

This is the single most important piece of context in the document, because roughly half the defects described below have a fix that exists in exactly one of the three places code can be.

```
$ TZ=UTC git log -10 --date=iso-local --pretty='%h %ad %s'
e35394a 2026-09-18 18:40:48 +0000 The last six labels that still called a ממשק a ממשק
bfc60db 2026-09-18 18:34:05 +0000 (availability, unrelated)
e89b926 2026-09-18 18:19:49 +0000 1.1.10 – and the date range that was never actually fixed
8235851 2026-09-18 18:14:22 +0000 Round 4: the blast radius, and the four the fourth reviewer ca
5531a21 2026-09-18 16:02:12 +0000 Seasons: the rest of the audit, and the six regressions the fi
bc6a562 2026-09-18 14:03:26 +0000 Seasons: the numbers people actually read
ae2c042 2026-09-18 13:53:12 +0000 Seasons: the six defects that made the feature unusable
4be7f1c 2026-09-18 11:38:18 +0000 A reopened season counts its own games, not a number it inheri
8a3b9ef 2026-09-18 11:34:55 +0000 The hall of fame becomes a ceremony
1672d6b 2026-09-18 06:01:07 +0000 A picker needs something to pick
4de80d0 2026-09-18 05:54:25 +0000 1.1.9 ← both store binaries were built HERE
```

The backend in production is `HEAD`. `git diff --stat 8235851 HEAD -- functions/` is empty, and every seasons Cloud Function reports an `updateTime` of 2026-09-18T18:16:53-57Z, two and a half minutes after `8235851` landed:

cronEvery60Min	2026-09-18T18:16:57Z	enableClubSeasons	2026-09-18T18:16:55Z
onGameRosterChanged	2026-09-18T18:16:55Z	reopenLastSeason	2026-09-18T18:16:55Z
onNotificationCreated	2026-09-18T18:16:54Z	endSeasonNow	2026-09-18T18:16:53Z
commitRoundStats	2026-09-18T18:16:54Z	updateSeasonTarget	2026-09-18T18:16:53Z

(disableClubSeasons and addRetroGoal show older timestamps because they did not change; Firebase skips unchanged functions.) The committed functions/lib/ is stale, but it is not what runs: firebase.json's predeploy compiles functions/src — so the stale build output is a repository-hygiene problem, not a production one.

The security rules in production are HEAD . Release cloud.firestore points at ruleset 70cde240-0d57-4070-a311-9b1df55a1598 , updateTime 2026-09-18T18:14:42Z . Its source is 114,528 bytes and diff s identical to firestore.rules at e35394a , including the season-archive participant clause's games > 0 || rounds > 0 . (An earlier audit round left this as an open question because of the 403 described above; it is answered, and the finding it fed is closed.)

The client in the stores is eleven commits behind. Both 1.1.9 artefacts — Android versionCode 235 and iOS build 103 — were built from 4de80d0 (2026-09-18 05:54Z). git diff --stat 4de80d0 HEAD -- src/ is 27 files, +3,240 / -467. Five files this document describes do not exist at all in the shipped binary: src/utils/seasonCardVariant.ts , src/utils/seasonMedalTier.ts , src/utils/seasonScope.ts , src/components/community/SeasonMedal.tsx , src/components/community/SeasonPoster.tsx . app.json says "version": "1.1.10" and no build of it exists; appConfig/android advertises 1.1.9 and appConfig/ios advertises 1.1.7 .

How to read a defect claim in this document

Marking	Means	Who can see it
live	present at HEAD, deployed, and reachable	everyone
fixed and deployed	server-side fix, in functions/src and running since 18.09 18:16Z	nobody any more — but the data an earlier version wrote is still there
HEAD only / not shipped	client-side fix, committed, in no store build	nobody yet; the defect is what users have today

The asymmetry is the trap: the server was fixed eleven times on 18.09 and redeployed the same evening; the client was fixed in the same commits and has not been built since. Where a single defect had halves on both sides — and several did — the server half is closed and the client half is live.

What a reviewer can re-check, and the three things they cannot

Almost every production figure here is a document a reviewer with owner credentials can fetch: the club document, the 23 game documents, the one seasonSummary , the one seasonCards , the 93 communityPlayerStats rows, the 717 communityPairStats rows, the seven notification documents, the seven users/*/seasonTitles documents. Where a figure is a count, it was produced by paging the whole collection, not by a limit .

Three claims are weaker than the rest and are marked where they appear:

1. **"24 of 30 clubs could never close a season"** (§5). Produced by an auditor porting `clubIsQuiet` out of the repository and running it over all finished games. Nothing in the repo reproduces it and no club ids were recorded. It is the largest adoption claim in this document and the one a reviewer cannot re-derive as written.
 2. **"two consecutive reads of the immutable archive returned different key orders"** (§1, §10). Asserted by an earlier audit round; the two orderings were not recorded. The *fix* (`seasonArchive.ts:114`) is real and checkable; the observation is not.
 3. Cloud Run log lines from the 17.09 close. They aged out of the 30-day window's early part and are not quoted anywhere in this document; every figure that was once sourced from a log line has been replaced with the document counts that support it.
-

Glossary

Term	Meaning in this codebase
מחזור (<i>machzor</i>)	one evening / game-night. A club plays one a week. The unit <code>completedRounds</code> , <code>playedRounds</code> , <code>eveningsSealed</code> , <code>roundsAtStart</code> and <code>targetRounds</code> are all measured in — despite three of those names saying "rounds".
מישחקון (<i>mischakon</i>)	one mini-game inside an evening, ~4–6 per night, recorded only in advanced mode. The unit <code>totals.rounds</code> , <code>players[].rounds</code> , <code>wins</code> , <code>goals</code> , <code>assists</code> and <code>cleanSheets</code> are measured in.
cadence	the rule that decides when a season is over: <code>{type:'rounds', targetRounds}</code> or <code>{type:'date', months, startsOn, endsOn, endsAt}</code> . Resolved to a <i>finish line</i> by <code>seasonFinishLine</code> .
seal	what happens to an evening when it finishes: <code>sealRoundSummary</code> writes <code>roundSummaries/{gameId}</code> , bumps <code>clubRecords.eveningsSealed</code> , and — if the evening belongs to the running season — increments <code>seasons.playedRounds</code> . Seasons are <i>closed</i> ; evenings are <i>sealed</i> .
close	the season ceremony: archive, card, titles, push, wind-back, open the successor. Three functions can start one; all end in <code>closeSeason</code> .
archive	<code>seasonSummary/{groupId}__{seasonId}</code> , written exactly once with <code>create()</code> : frozen player rows, pair rows, club totals and the nine award decisions. The only copy — a reopen deletes it.
card	<code>seasonCards/{groupId}__{seasonId}</code> , a ~2.5 KB projection of the archive (date, three totals, participant count, winner names) that the hall of fame lists instead of pulling every archive.
wind-back	the subtraction the close performs on the live rollup rows: <code>live - archived</code> , never an absolute zero, so an evening committed mid-close is not destroyed. The club totals document breaks this rule; see §5.
stamp	<code>games/{gameId}.seasonId</code> , the write-once string that says which season an evening belongs to. Absent means season 1.
the three counters	<code>clubRecords.eveningsSealed</code> (lifetime, server-only), <code>seasons.playedRounds</code> (per-season mirror, client-readable), and the client's 200-document games scan. See §3.
LEDGER / ROUND4	the two audit records this document folds in: round 3 (~130 findings, referenced by slug, e.g. <code>mvp-title-crowns-the-entire-club</code>) and round 4 (30 findings, referenced by number, e.g. <code>ROUND4#13</code>).

Contents

§	Section	What it answers
1	The data model	every document and field a season touches, who writes it, what absence means
2	Turning seasons on, off, and on again	activation, the two history choices, the seeding, the refusals
3	How a מחזור is counted	the three counters, the four server derivations, the stamp
4	Cadence — how a season knows it is over	rounds and date finish lines, the calendar arithmetic, moving the line
5	Closing a season	the three entry points, <code>closeSeason</code> step by step, the wind-back
6	Undoing a close	what the reopen restores, what it destroys for ever, the 48-hour grace
7	Where the numbers come from	the two functions that write every archived counter, and everything that mutates an evening after the fact
8	The nine titles	the award algorithm, the eligibility gate, the medals
9	Season scope versus lifetime	how the statistics are divided, and the five callers of the scan
10	The personal season summary	the per-player screen, its two load paths, its ranks and peers
11	The screens	every surface, pixel by pixel, and the Hebrew it prints
12	Notifications, rules, triggers and cost	the push nobody received, the rules, the sweep, the read bill
13	What is tested, what is not, and the open questions	coverage, the guards, and eight decisions that are genuinely open

1. The data model

Scale, first, because it changes how you read everything else

Collection	Documents in production
groups	195
groups with a seasons block at all	2
groups with seasons.enabled == true	1 (HhzIwmjMl1i5HS0GHt3p , מועדון שכחת שושי , "Shoshi's Forgetting Club")
seasonSummary	1 (HhzIwmjMl1i5HS0GHt3p__s1)
seasonCards	1 (same season)
roundSummaries for that club	10, against 23 finished games
communityPairStats for that club	314, of which 293 are guest pairs

There is exactly one archived season in the entire database. Every "worked example" below is that season, because there is no other. The feature has never run anywhere else, which is why so many defects survived to production: the code paths that only fire on season 2, 3 and 4 have literally never executed on real data.

groups/{groupId}.seasons — the only mutable lifecycle record

This sub-map is the whole state machine. It is the only seasons data a client may read cheaply, and it is the only seasons data the client may not write: the rules lock the field, and every mutation goes through one of five callables (enableClubSeasons , disableClubSeasons , updateSeasonTarget , endSeasonNow , reopenLastSeason) or two server paths (performSeasonClose at functions/src/index.ts:15884–15925 , and the per-seal increment at functions/src/index.ts:5219–5232).

The declared type is GroupSeasons , src/types/index.ts:959–1000 :

```

export interface GroupSeasons {
  enabled: boolean;
  currentNo: number;
  currentId: string;
  startedAt: number;
  roundsAtStart?: number;
  playedRounds?: number;
  cadence: {
    type: SeasonCadenceType; // 'date' | 'rounds'
    months?: number;
    endsAt?: number;
    startsOn?: string; // 'YYYY-MM-DD', club calendar
    endsOn?: string; // 'YYYY-MM-DD', LAST valid day
    targetRounds?: number; // TOTAL finished evenings, not a remainder
  };
  targetHistory?: SeasonTargetChange[];
  count: number;
}

```

The live production document:

```

{
  "enabled": true,
  "currentNo": 2,
  "currentId": "s2",
  "startedAt": 1789679141952, // 2026-09-17 21:05:41Z
  "roundsAtStart": 10,
  "playedRounds": 0,
  "reopenedAt": 1789727847680, // 2026-09-18 10:37:27Z
  "count": 1,
  "cadence": { "type": "rounds", "targetRounds": 24,
    "months": null, "endsAt": null, "endsOn": null, "startsOn": null },
  "targetHistory": [ { "at": 1789727874911, "by": "YIZlKWbvvjae3oqgIoAMr9nzQEi1",
    "byName": "Eliran Tzabari",
    "from": { "type": "rounds", "targetRounds": 2, ... },
    "to": { "type": "rounds", "targetRounds": 24, ... } } ]
}

```

Field by field:

Field	Type	Written by	Read by	Absent means	Absent ≠ 0?
enabled	bool	all five callables	everything; the sweep's query is <code>where('seasons.enabled','==',true)</code> (<code>index.ts:16078</code>)	feature off	n/a
currentNo	int, 1-based	enable / close / reopen	id derivation, <code>eveningInSeason</code> fallback	treated as 1 by the reader	no
currentId	string <code>s{N}</code>	same	the stamp written onto games; the scope filter; the archive doc id	the client reader returns <code>undefined</code> for the whole block (<code>firestore.ts:671</code>)	n/a
startedAt	epoch ms	<code>close(now)</code> , <code>enable(now)</code> , reopen (archive's <code>startsAt</code>)	hall-of-fame date range, <code>rebaseCadence</code>	0	no
roundsAtStart	int	<code>seasonSeed()</code> = <code>clubRecords.eveningsSealed</code> at open	<code>completedRoundsFrom</code> fallback; <code>roundSummaries.seasonEvenings</code>	0	no
playedRounds	int	seeded at open, then <code>FieldValue.increment(1)</code> per seal	the card, the sweep, <code>endSeasonNow</code> , the running personal summary	falls back to <code>eveningsSealed</code> – <code>roundsAtStart</code>	yes — <code>completedRoundsFrom(seasonCounters.ts:110)</code> tests <code>typeof playedRounds === 'number'</code> , so 0 is honoured and absent is not
cadence.type	'date' \ 'rounds'	enable / retarget / <code>rebaseCadence</code>	<code>seasonFinishLine</code>	client reader defaults to 'date' (<code>firestore.ts:695</code>)	no
cadence.months	int	enable (length) / retarget (remaining)	<code>rebaseCadence</code>	no length recorded	no
cadence.endsAt	epoch ms	legacy date seasons	<code>seasonFinishLine</code> kind 'epoch'	prefer <code>endsOn</code>	yes (null vs absent both falsy)

Field	Type	Written by	Read by	Absent means	Absent ≠ 0?
<code>cadence.startsOn / endsOn</code>	YYYY-MM-DD	date cadence	<code>seasonFinishLine</code> kind 'date', DST-safe	no calendar boundary	yes
<code>cadence.targetRounds</code>	int	enable / retarget	<code>seasonFinishLine</code> kind 'rounds'	<code>{kind: 'none'}</code> → a season that can never end	yes
<code>targetHistory</code>	array	<code>updateSeasonTarget</code> via <code>arrayUnion</code> ; reset to <code>[]</code> by every open	nothing on any screen	never moved	<code>[]</code> and absent are indistinguishable
<code>count</code>	int	+1 on close, -1 on reopen	which season the undo button targets; whether enable seeds history	0	no

Three further fields are written by the server and **do not exist in the TypeScript type, are not named by the client deserializer, and therefore do not exist on any client**:

- `reopenedAt: number` — stamped by `reopenLastSeason` (`index.ts:16905`), zeroed by every other opener. Grants a 48-hour immunity from re-closing (`REOPEN_GRACE_MS` , checked at `index.ts:15979` and `index.ts:16179`). The live club carries `reopenedAt = 2026-09-18 10:37Z` right now.
- `targetMovedToClose: {at, by, byName}` — set by `updateSeasonTarget` when the new target is already behind the club (`index.ts:16815`), read by `performSeasonClose` to stamp `endedEarly` .
- `dueBlockedSince: number` — set when a season came due and could not be closed (`index.ts:16206`), deleted when it finally is.
- `seasonStampRetry` is the same class of problem one collection over (see games, below).

`src/firebase/firestore.ts:667-714` rebuilds the block field by field, so **a field the reader does not name does not exist on the client, whatever Firestore holds**. This is a recurring failure mode in this codebase, and `targetHistory` was its most recent victim: the server wrote it on every change, the type documented it as "kept and shown... never quietly", and the reader dropped it. The reader was fixed; nothing renders it even now (ledger `target-history-never-shown-to-anyone`).

`SeasonTargetChange` (`src/types/index.ts:1005-1013`) is also already behind its writer. The writer at `index.ts:16802-16812` adds two more fields per entry:

```
targetHistory: admin.firestore.FieldValue.arrayUnion({
  at: now, by: uid, byName,
  from: seasons.cadence ?? null,
  to: next,
  playedAtMove: played,      // not in the type
  endsTheSeason,             // not in the type
}),
```

and no entry carries a `seasonId` . That omission is what allows ledger finding `reopen-inherits-the-successor-seasons-target-history` : an array element cannot say which season it belongs to, so a reopen that restores the wrong array cannot be detected.

`games/{gameId}.seasonId` — the membership stamp

One optional string field on the game document. Written once by `onGameRosterChanged` (`functions/src/index.ts:5802-5836`) on the `→ active` or `→ finished` transition, guarded by `!after.seasonId` so it is effectively write-once. A failed group read writes `seasonStampRetry: true` (`index.ts:5842`) so the next write to the game tries again.

The rule that reads it exists in two copies that are deliberately identical — `inSeason` at `src/utils/seasonScope.ts:28` (client) and `eveningInSeason` at `functions/src/seasonCounters.ts:53` (server), quoted side by side in §3. Both reduce to one line: a stamped evening belongs to the season whose id it carries, and an **unstamped** evening belongs to season 1.

Absence is load-bearing and means "season 1". That is the whole of "your history becomes season 1". It is also, structurally, a one-shot promise: it can only ever be told once per club, and it is wrong for any club that disables and re-enables the feature.

Every finished game of the only club that runs seasons:

Evenings	seasonId
19 evenings, 2026-06-28 → 2026-09-15	<i>absent</i> → resolved to season 1
3 evenings, 2026-09-16 → 2026-09-17	"s1"
1 evening, 2026-09-17 21:23 (DTNscolR...)	"s3"
any	"s2" — none

The last row is a dead stamp. Season 3 was opened when season 2 closed at 21:12Z, the game was stamped `s3` eleven minutes later, and then season 2 was reopened on 2026-09-18, which decrements `count` and deletes `s2`'s archive — so `s3` now names a season that does not exist and will not exist again until the club closes `s2`. `eveningInSeason(game, 's2', 2)` is false (stamp present, mismatched), so that evening belongs to **no** season on either side of the wire: it is in no archive, in no live scope, and `seasons.playedRounds` is 0 despite it having been played inside `s2`'s window. The stamp is write-once, finished games are client-read-only, and the

trigger only fires on a status transition a finished game has no more of — so there is no repair path. Ledger: `undo-close-orphans-the-season-stamps` , `orphan-season-stamp-absorbed-by-the-next-season-of-the-same-id` .

`seasonId` appears **nowhere in** `firestore.rules` (verified by grep: the only two hits are comment lines 2097 and 2111, both about other collections). A club member may therefore create a game pre-stamped with any season id, and the server's write-once check then refuses to correct it (ledger `games-seasonid-unpinned` , emulator-proven with four successful writes).

The three live tables a close winds back

These are the club's running counters. They are not seasons documents — they predate the feature — but a season close subtracts its archive out of them and a reopen adds it back, so their exact field sets are part of the seasons contract. The lists are pinned in one place, `functions/src/seasonRollover.ts:184-242` (condensed here; the file lists one field per line, each with the reasoning for its presence):

```
const PLAYER_SEASON_FIELDS = [                // 19 fields, :184-211
  'goals','assists','rounds','wins','losses','ties','games','cleanSheets','ownGoals',
  'penTaken','penScored','penMissed','penFaced','penSaved','penConceded',
  'csRounds','asRounds',                      // coverage denominators
  'eveningScoreSum','eveningScoreCount',
] as const;
const PAIR_SEASON_FIELDS = [                  // 10 fields, :220-231
  'assists','sameTeam','winsTogether','lossesTogether','cleanSheetsTogether',
  'against','winsA','winsB','assistsAToB','assistsBToA',
] as const;
const CLUB_SEASON_FIELDS = [                  // 7 fields, :234-242
  'rounds','goals','guestGoals','ownGoals','tiedRounds','shootoutRounds','scorelessRounds',
] as const;
```

Anything **not** on these lists survives a close: `bestEvening` , `lastEveningScore` , `kingGoalsSum` / `kingGoalsCount` on player rows, `chemistrySince` and the `king*` accumulators on the club row. The module comment at the top of `seasonRollover.ts` explains why — resetting `kingGoalsSum` would make the first top scorer of every new season a perfect 10 by construction.

`communityPlayerStats/{groupId}__{uid}` . Doc id is the composite; `groupId` and `userId` are also fields, because a `list` does not bind the path wildcard (`firestore.rules:2070`). It carries **no display name** — that is why `closeSeason` issues a separate `getAll` over `/users` (`seasonRollover.ts:317-333`) to freeze names into the archive. Three season stamps live here:

- `seasonWoundBack`: `string` — written by the close, cleared by the reopen. Makes the subtraction idempotent.
- `seasonReopened`: `string` — written by the reopen, deleted by the close. Makes the addition idempotent.
- Both are read-only signals; neither is in any client type.

The real row for `מִתָּן לוי` (`1IdtNEjbEXfiRSqvLrJVn99Nsfi2`) today, trimmed:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p", "userId": "1IdtNEjbEXfiRSqvLrJVn99Nsfi2",
  "goals": 0, "assists": 0, "rounds": 0, "wins": 0, "losses": 0, "ties": 0, "games": 0,
  "cleanSheets": 0, "csRounds": 0, "asRounds": 0,
  "eveningScoreSum": 0, "eveningScoreCount": 0,
  "bestEvening": { "goals": 0, "assists": 0, "involvement": 0, "cleanSheets": 2, "wins": 2 },
  "lastEveningScore": 6,
  "seasonWoundBack": "s1",
  "updatedAt": 1789661526768 }
```

Every season-owned counter is zero. That is correct behaviour — the season was archived — and it is also the single fact behind the owner's original complaint: the club screen scans games and says 22 evenings, and every screen that reads this row says 0 (ROUND4 §2, ledger `evenings-22-on-one-screen-0-on-the-next`). Note the stamp still reads `s1` even though `s2` has since been closed and reopened; the `s2` close skipped this row because a row with `rounds == 0 && games == 0` is not archived as a participant (`seasonRollover.ts:366`).

`communityPairStats/{groupId}__{a}__{b}` . Live `a / b` are *not* sorted; the archive key is `seasonPairKey()` (`seasonRollover.ts:104–110`) which sorts them and returns a `flip` flag so the directional counters (`winsA` , `assistsAToB`) can be re-oriented. A live row today:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p",
  "a": "CEKRfDs20xSuiW6xmPKlMcx3NoV2", "b": "JoLFRxFr0tTaYfVAXBPwkvT0et62",
  "sameTeam": 0, "against": 0, "winsTogether": 0, "lossesTogether": 0,
  "cleanSheetsTogether": 0, "winsA": 0, "winsB": 0,
  "assists": 0, "assistsAToB": 0, "assistsBToA": 0,
  "seasonWoundBack": "s2", "updatedAt": 1789679528192 }
```

`seasonWoundBack: "s2"` while **s2 is the running season**. All 314 pair rows read this. The stamp is the close's idempotency latch, so the next close of `s2` will archive this pair's chemistry *and* skip winding it back — sealing it into a write-once archive while leaving it standing live for `s3` to inherit, double-counted. That is what would happen against the code as it stood when the reopen ran. It is **not** what will happen now: the `resuming && qualifier` that makes the stamp evidence only on a resume (`seasonRollover.ts:886` , `:973`) was deployed at 18:16Z on 18.09, so the next close of `s2` winds these rows back correctly. The clearing sweep at `seasonRollover.ts:1174–1198` is deployed too, but it runs only on a *reopen*, and the reopen that armed this state ran seven and a half hours before it existed. Ledger: `stale-pair-woundback-stamp-skips-next-wipe` , ROUND4 §9 — the defect is closed in production; only the anomalous data remains. See §6.

`assists` versus `assistsAToB` / `assistsBToA` is a real ambiguity, not a duplication: the legacy undirected `assists` counts a **wider window** than the directional pair, so the two disagree. The archive keeps both; the awards use directional only (`seasonRollover.ts:527`), the club chemistry card uses legacy, and `seasonArchive.topDuo` (client) uses directional. On `s1` they are 7 and 8.

`communityStats/{groupId}` . One doc per club, `id = groupId`, with `groupId` also as a field for the same wildcard reason. Season-owned: the seven `CLUB_SEASON_FIELDS` . Not season-owned: `kingGoalsSum/Count` , `kingAssistsSum/Count` , `chemistrySince` . The close **sets the seven to absolute 0**

(`seasonRollover.ts:1004–1021`) while the player and pair rows are *subtracted* — an inconsistency the module's own header calls out as forbidden ("SUBTRACT, do not write zeroes") and then violates fifty lines from the bottom. Ledger `club-totals-zeroed-not-subtracted` . Live:

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p", "rounds": 0, "goals": 0, "guestGoals": 0, "ownGoals": 0,
  "tiedRounds": 0, "shootoutRounds": 0, "scorelessRounds": 0,
  "kingGoalsSum": 6, "kingGoalsCount": 5, "kingAssistsSum": 4, "kingAssistsCount": 3,
  "chemistrySince": 1789679528192, "seasonReopened": "s2", "updatedAt": 1789727846661 }
```

`clubRecords/{groupId}` — `eveningsSealed` , and why nobody can read it

```
match /clubRecords/{groupId} { allow read, write: if false; } // firestore.rules:1813
```

Server-only in **both** directions, by design: it is the baseline the evening summary measures records against, and a second readable copy would become a drifting source of truth for "the best evening ever". The consequence for seasons is concrete: the counter a rounds target is measured against is unreadable by the app, which is the entire reason `groups.seasons.playedRounds` exists as a client-readable mirror (`index.ts:5210–5232`).

```
{ "groupId": "HhzIwmjMl1i5HS0GHt3p",
  "eveningsSealed": 10,
  "since": 1787739600000, // 2026-08-26 10:20Z
  "lastEveningAt": ..., "updatedAt": 1789655711795,
  "goals": {"value":2,"userIds":[...]}, "assists": ..., "wins": ..., "cleanSheets": ..., "involvement": ...,
  "firstEverSeen": ["every_team_won","multiple_personal_records"] }
```

`eveningsSealed` is `FieldValue.increment(1)` per sealed evening (`index.ts:5194`), atomic specifically because an absolute write lost evenings when three sealed within a minute. But **it began on 2026-08-26**. The club has played 23 evenings and the counter says 10. Any arithmetic that subtracts `roundsAtStart` from it — `completedRoundsFrom` 's fallback (`functions/src/seasonCounters.ts:111–117`), `roundSummaries.seasonEvenings` (`index.ts:5157`) — is answering for the era of the counter, not for the club. On this club that fallback yields $10 - 7 = 3$ for a season the card says held 22. That is the arithmetic that `endSeasonNow` used to perform (two arguments instead of three) and would have sealed into a write-once archive.

`seasonSummary/{groupId}__{seasonId}` — the archive

Written exactly once, with `create()` (`seasonRollover.ts:584`), which is the feature's entire idempotency story. Deleted only by `reopenSeason` . Two read statements in the rules (`firestore.rules:2121–2164`): members list it by field; a **participant** — a uid that is a key in the `players` map with `games > 0 || rounds > 0` — may get it for ever, even after leaving the club.

Top-level shape:

Field	Type	Present when	Read by
groupId , seasonId , no	string/int	always	everything
groupName	string	always (frozen)	personal summary, for readers who can no longer read /groups
startsAt	epoch	always	hall of fame date range; reopenLastSeason restores seasons.startedAt from it
endsAt , closedAt	epoch	always, both = now	display
completedR ounds	int	always	the season's length. reopenLastSeason fallback; seasonSummaryService.fromArchive (:382)
roundsAtSt artOfSeaso n	int	always	server only — reopenLastSeason:16893 restores the offset
zeroedAt	epoch	only after the wipe finished	server only — distinguishes "already closed" from "resume a half-finished close" (seasonRollover.ts:667)
endedEarl y	true	omitted unless set	seasonHistoryService:89 , badge on the card
partialDat a	true	omitted unless set	SeasonHistoryScreen:286 , the "נתונים חלקיים" (partial data) chip
closedBy , closedByNa me	string	manual close only	nothing — write-only, costs one extra /users read per close
originalTa rget	cadence map	when the caller passed one	reopenLastSeason:16955 re-bases the cadence from it
targetHist ory	array	omitted when empty	nothing
totals	map	always	club table
players	map uid → row	always	everything
pairs	map lo__hi → row	always	duo title, personal summary peers
awards	map of 9 keys → {winners:string[], value:number} \ null	always	card, titles, personal summary

The production document, trimmed:

```
{ "groupId": "HhzIwmjMl1i5HSOGHt3p", "groupName": "מועדון שכחת שושי",
  "seasonId": "s1", "no": 1,
  "startsAt": 1789547794676, // 2026-09-16 08:36:34Z
  "endsAt": 1789661526768, // 2026-09-17 16:12:06Z
  "closedAt": 1789661526768,
  "zeroedAt": 1789661533425,
  "completedRounds": 22,
  "roundsAtStartOfSeason": 7,
  "originalTarget": { "type": "rounds", "targetRounds": 22 },
  "totals": { "rounds": 37, "goals": 27, "assists": 13, "cleanSheets": 75,
    "guestGoals": 4, "ownGoals": 0, "tiedRounds": 9,
    "shootoutRounds": 6, "scorelessRounds": 16 },
  "awards": { "topScorer": {"winners":["alsobLSA..."],"value":10},
    "topAssister": {"winners":["K5rSGB4J..."],"value":5},
    "mvp": {"winners":[ ... all seven uids ... ],"value":6},
    "topWinner": {"winners":["B5KpY04I..."],"value":13},
    "mostLoyal": {"winners":["1IdtNEjb..."],"value":19},
    "cleanSheetKing": {"winners":["alsobLSA..."],"value":13},
    "penaltyKing": {"winners":["alsobLSA..."],"value":1},
    "penaltyKeeper": {"winners":["alsobLSA..."],"value":1},
    "deadlyDuo": null } }
```

Read that document against the club's own history and three things are visibly wrong:

1. **startsAt is the moment the admin switched the feature on**, not the season's first evening. The season contains 22 evenings from 2026-06-28; the archive says it began on 2026-09-16 and ended on 2026-09-17. **endsAt – startsAt** is 31.6 hours. The shipped 1.1.9 hall of fame renders this as "2026 ספט – 2026 ספט" ("Sep 2026 – Sep 2026") above "22 מהזורים" (22 evenings); at HEAD the same-month collision falls through to days and it reads "16 (11§) 17 בספט – בספט", which is a more precise caption for the wrong event. Ledger **season1-startsat-is-the-enable-moment**, **season1-range-prints-a-31-hour-lie**. Worse, the client's **VOID_SPAN** guard treats a sub-48h season as a bookkeeping artefact; only **players !== 0** stops this club's entire hall of fame collapsing into a grey dashed line.
2. **completedRounds (22) and the awards disagree**. The awards were decided against a *different* denominator — **awardsDenominatorOf(players.games) = max(games) = 19** (**seasonRollover.ts:133–139, 556**). **minPenaltyAttempts(19) = 2** but **minPenaltyAttempts(22) = 3** (**src/utls/seasonAwards.ts:126**). **הלן צברי** took exactly 2 penalties and scored both; Nofar took 3 and scored 2. At 19 the title is **הלן's** on a rate of 1.0; at 22 **הלן** is not eligible and the title is Nofar's. The archive as it stands publishes the 22 and the 19-derived winner side by side, for ever, because **create()** makes it write-once and nothing recomputes. Ledger **awards-denominator-derivation-untested**, **repaired-completedrounds-silently-contradicts-the-sealed-titles**. The **updateTime** of this document (2026-09-18 11:41Z) is a day after its **createTime** (2026-09-17 16:12Z) — the length was hand-repaired; the awards were not.

3. `partialData` is **absent** on a season that is unambiguously partial: 19 of its 22 evenings predate the feature, assists were only collected from 21.06 and clean sheets from 17.08. The flag is written only on the `sealNow` activation branch (`index.ts:16483`), never on a rollover close and never on `endSeasonNow` . Three independent agents found this. Ledger `partialdata-only-on-the-sealnow-path` .

Also note `mvp.winners` has **seven of seven** members at `value: 6` . `eveningScoreCore` floors the scale at 6.0 and also returns 6.0 as its "no data" sentinel; every archived row has `eveningScoreSum / eveningScoreCount` exactly 6.0 (12/2 and 18/3). `leaders()` uses a floor of 0, so the whole club won שחקן העונה ("player of the season"). This is verifiable from a third source: a collection-group query over `users/*/seasonTitles` returns seven `..._s1__mvp` documents.

THE `PLAYERS` MAP ROW — AND THE ONE PLACE ABSENCE IS PRESERVED

```
"1IdtNEjbEXfiRSqvLrJVn99Nsfi2": {
  "displayName": "מתן לוי",
  "games": 19, "rounds": 26, "goals": 2, "assists": 1,
  "wins": 11, "ties": 6, "losses": 9,
  "cleanSheets": 12, "ownGoals": 0,
  "penTaken": 1, "penScored": 0, "penMissed": 1, "penFaced": 1, "penSaved": 1, "penConceded": 0,
  "csRounds": 15, "asRounds": 24,
  "eveningScoreSum": 18, "eveningScoreCount": 3
}
```

`games` counts **evenings** (מחזורים); `rounds` counts **mini-games** (משחקונים). Both are in the same row with adjacent names and the code conflates them constantly. All seven rows here have `games` of 17–19 and `rounds` of 21–27. For a club on the plain timer — the common case — `rounds` would be 0 for every player of every season, which is why 23 of production's 93 stat rows have `games > 0` and no `rounds` at all.

Two fields, and only two, deliberately preserve the difference between absent and zero (`seasonRollover.ts:351–356`):

```
for (const f of PLAYER_SEASON_FIELDS) {
  // ABSENT is not zero for the two coverage denominators.
  if ((f === 'csRounds' || f === 'asRounds') && typeof x[f] !== 'number') continue;
  row[f] = num(x[f]);
}
```

and the reader mirrors it (`src/utils/seasonArchive.ts:102–103`):

```
...(typeof x.csRounds === 'number' ? { csRounds: x.csRounds } : {}),
...(typeof x.asRounds === 'number' ? { asRounds: x.asRounds } : {}),
```

The reason is arithmetic, not taste. `csRounds` and `asRounds` are *coverage denominators*: how many mini-games the clean-sheet and assist metrics were actually being recorded in. They arrived later than the metrics they divide. A stored `0` asserts "measured across zero rounds", which is a different statement from "we do not

know how many rounds this was measured across" — and the efficiency tab's fallback (divide by `rounds` instead) only fires on the second. Writing 0 made every long-standing player's clean-sheet percentage read about ten points low. Every *other* field in the archive defaults to 0 on read (`num()` at `seasonArchive.ts:9`), which is the ledger finding `older-archive-degrades-into-plausible-zeros`: an archive written by an older build yields confident wrong numbers rather than blanks, with no version field anywhere to detect it.

THE PAIRS MAP

Keyed "`<lo>__<hi>`" by sorted uid, so `a / b` and every directional counter are re-oriented on the way in. Guests (`id.startsWith('guest:')`) are dropped, and so is any pair whose counters sum to zero; the cap is `MAX_ARCHIVED_PAIRS = 1200` (`seasonRollover.ts:181`), chosen to keep the document under Firestore's 1 MB limit. The documents reconcile exactly: the close read **314** pair rows (paged live today), the archive's `pairs` map holds **21**, and the other **293** are guest pairs it refused. Nothing deleted those 293 until the guest-pair wipe that landed on 18.09 and has not yet had a close to run in.

```
"CEkRfDs20xSuiW6xmPKLMcx3NoV2__JoLFRxFr0tTaYfVAXBPwkvT0et62": {
  "a": "CEkRfDs...", "b": "JoLFRxFr...",
  "sameTeam": 6, "against": 6,
  "winsTogether": 1, "lossesTogether": 2, "cleanSheetsTogether": 4,
  "winsA": 4, "winsB": 2,
  "assists": 0, "assistsAToB": 0, "assistsBToA": 0 }
```

Across all 21 archived pairs the directional assists sum to 7 and the legacy `assists` to 8, against `totals.assists = 13`. So the pair counters account for barely half the season's assists, and the largest directional pair total is 2 — below `MIN_DUO_ASSISTS = 3` (`src/utils/seasonAwards.ts:137`), which is why `awards.deadlyDuo` is `null`. The archive *reader's* `topDuo` (`seasonArchive.ts:128–155`) has **no** floor, so the stats screen happily names a duo on 2 assists that the season refused to crown, with ties broken by Firestore map-iteration order. Ledger `archive-duo-reader-has-no-floor`, `assist-peers-account-for-half-the-season-assists`.

One more property of this document worth naming: **the players map has no order**. `Object.entries` over a Firestore map returns whatever the wire gave it, and an earlier audit round reported two consecutive reads of this immutable document coming back in different orders — an observation whose two orderings were unfortunately not recorded, so treat it as asserted rather than demonstrated. The guarantee is absent either way: nothing in the protocol promises map order. `parseSeasonTable` now pins `players.sort((a,b) => a.uid.localeCompare(b.uid))` (`seasonArchive.ts:114`), which is real and checkable — and is **not** in either shipped 1.1.9 binary (ROUND4 §1). See §10 for what the un-ordered version does to a player's rank.

`seasonCards/{groupId}__{seasonId}` — the list row

Same close, same numbers, one hundredth the size, written with `merge: true` so a resume is a no-op (`seasonRollover.ts:752–809`). The hall of fame reads only this; the archive is pulled lazily when a season is opened. Read rule mirrors `seasonSummary`'s member clause but has **no** participant clause — a departed player can open the archive and not the list.

```
{ "groupId": "HhzIwmjMl1i5HSOGht3p", "seasonId": "s1", "no": 1,
  "startsAt": 1789547794676, "endsAt": 1789661526768,
  "completedRounds": 22,
  "totals": { "rounds": 37, "goals": 27, "assists": 13 },
  "players": 7,
  "winners": [ {"key":"topScorer","names":["הלן צברי"],"value":10},
               {"key":"mvp","names":["מתן לוי","Lioz Madar","Linoy Levi","איציק לוי",
                                     "Nofar Tzabari","Eliran Tzabari","הלן צברי"],"value":6},
               {"key":"mostLoyal","names":["מתן לוי"],"value":19}, ... ] }
```

players: 7 comes from `countSeasonParticipants`, which counts rows with `games > 0` and falls back to `rounds > 0`. `totals.rounds: 37` is **mini-games**, sitting one field away from `completedRounds: 22` which is **evenings**; the Hebrew copy renders the first as "37 משחקים" (games) and the second as "22 מחזורים" (evenings), forty points apart on the same card. `winners` carries names, not uids, and they are frozen — which is correct for a record and is also why a rename breaks the title-streak logic that joins on them.

The card does **not** carry `awardsDenominator`. It did for one day; because the field is `max(players.games)` and `mostLoyal` is the maximum of the same array, `value / denominator` was 1.0 identically and the loyalty medal drew platinum for every season of every club. The field is gone from both writer and card; see `awardsDenominatorOf`'s doc comment (`seasonRollover.ts:112–132`) and `ROUND4 §5`.

`users/{uid}/seasonTitles/{groupId}__{seasonId}__{titleKey}`

The public copy of a title, on the winner's profile. Deterministic id so a redelivery overwrites rather than duplicates; the duo title's joined key `a__b` is split and written to both profiles (`seasonRollover.ts:832–859`). Deleted exactly by `reopenSeason`, `seasonRollover.ts:1217–1245`. Rules: `allow read: if isSignedIn()`; `allow write: if false`; (`firestore.rules:2010`).

```
// users/1IdtNEjbEXfiRSqVlrJVn99Nsfi2/seasonTitles/HhzIwmjMl1i5HSOGht3p__s1__mostLoyal
{ "groupId": "HhzIwmjMl1i5HSOGht3p",
  "groupName": "מועדון שכחת שושי", // frozen: survives a club rename or the player leaving
  "seasonId": "s1", "seasonNo": 1,
  "titleKey": "mostLoyal", "value": 19,
  "at": 1789661526768 }
```

`value` is the raw winning number and its unit varies by title: 19 evenings for `mostLoyal`, 10 goals for `topScorer`, 13 mini-game wins for `topWinner`, an average of 6.0 for `mvp`, a *rate* of 1.0 for `penaltyKing`. Nothing on the document says which. The type is `SeasonTitle` (`src/types/index.ts:1029–1038`).

`roundSummaries/{gameId}` **and** `eveningStandings/{gameId}__{uid}`

Neither is a seasons collection, but both are inputs to the seasons machinery.

`roundSummaries/{gameId}` is created once per sealed evening with `create()` (`index.ts:4876` , `:5161`). Its relevance here is twofold. First, **its existence is the sweep's liveness gate**: `clubIsQuiet` (`index.ts:15466` , the unsealed-game check at `:15533–15570`) blocks a close when one of the three newest finished games has no `roundSummaries` document and `didEveningHappen` says it happened. The collection only began on 2026-08-26 — the live club has 10 summaries against 23 finished games — and a legacy close is 'happened'. On the numbers an earlier audit round produced, that permanently blocked most of the clubs in the database; §5 sets out the mechanism and the provenance of that figure. Second, it carries an optional `seasonEvenings` field derived as `eveningsSealed + 1 - seasons.roundsAtStart` (`index.ts:5157`) — a *sixth* answer to "how many evenings", from the counter that started late. No production document carries it; all ten predate the field.

```
// roundSummaries/TZ7IWEJWVw2itvIKCd8
{ "gameId": "TZ7IWEJWVw2itvIKCd8", "groupId": "HhzIwmjMl1i5HSOGHt3p",
  "at": 1789655700000, "generatedAt": 1789655711641, "version": 1,
  "basis": { "since": 1787739600000, "eveningsCompared": 9 },
  "coverage": { "hasRoundHistory": false, "hasAssists": false },
  "stats": { "rounds": 0, "goals": 0, "assists": 0, "ties": 0, "shootouts": 0 },
  "events": [ { "type": "club_milestone", "metric": "evenings", "threshold": 10, "total": 10 } ],
  "leaders": { "topScorers": null, "topAssisters": null, "topWinners": null,
    "topCleanSheets": null, "topGoalInvolvement": null },
  "pairHighlight": null, "teamHighlights": { "best": [], "worst": [] } }
```

`eveningStandings/{gameId}__{uid}` is a per-player, owner-read-only row (`firestore.rules:1772`): the score, its delta, and the rank before and after. Seasons touch it only by resetting the table it ranks against; `at` is stamped `Date.now()` rather than the evening's `startsAt` , so a late confirmation dates the row by the confirmation (ROUND4 §30).

Seven answers to one question

The reader will meet the phrase "how many מחזורים" in seven different places. They are different numbers and they disagree in production **today**:

#	Where	Derivation	Value on the live club
1	<code>clubRecords.eveningsSealed</code>	<code>increment(1)</code> per seal, since 26.08.2026	10
2	<code>groups.seasons.playedRounds</code>	seeded at open, <code>increment(1)</code> per seal of the running season	0 (s2)
3	<code>completedRoundsFrom</code> fallback	<code>eveningsSealed - roundsAtStart</code>	3
4	client games scan, <code>inSeason + didEveningHappen</code>	22 for s1, 0 for s2	22 / 0
5	<code>seasonSummary.completedRounds</code>	whatever the closing caller passed	22 (hand-repaired)
6	<code>awardsDenominatorOf = max(players.games)</code>	the eligibility gate's denominator	19

#	Where	Derivation	Value on the live club
7	<code>seasonCards.totals.rounds</code>	mini-games, mislabelled in Hebrew as games	37

`functions/src/seasonCounters.ts` was extracted specifically to make 1–4 testable side by side (`tests/logic/seasonCounterReconciliation.test.ts`), and it is the right instinct. But 5, 6 and 7 live in `seasonRollover.ts` and are never reconciled with 1–4, and only 5 and 6 decide anything irreversible.

Absence semantics, collected

Field	Absent means	Distinguishable from 0?	Who relies on it
<code>games.seasonId</code>	season 1	n/a (string)	<code>inSeason</code> , <code>eveningInSeason</code> — both sides
<code>seasons.playedRounds</code>	fall back to the sealed-counter subtraction	yes, tested with <code>typeof</code>	<code>completedRoundsFrom:110</code>
<code>seasons.roundsAtStart</code>	0	no	the subtraction fallback
<code>seasons.cadence.targetRounds</code>	<code>{kind:'none'}</code> → never due	yes	<code>seasonFinishLine:148</code>
<code>seasons.targetHistory</code>	never moved	no (<code>[]</code> ≡ absent)	nothing reads it
<code>seasons.reopenedAt</code>	no grace period	no	<code>index.ts:15979</code> , <code>:16179</code>
<code>seasonSummary.zeroedAt</code>	resume a half-finished close	yes (truthiness of a timestamp)	<code>seasonRollover.ts:667</code>
<code>seasonSummary.endedEarly / partialData</code>	not early / not partial	yes (omitted, not <code>false</code>)	card badges
<code>players[uid].csRounds / asRounds</code>	coverage unknown → divide by rounds instead	yes, deliberately	<code>seasonArchive.ts:102</code> , efficiency tab
every other archive field	0	no	<code>num()</code> fallback, <code>seasonArchive.ts:9</code>
<code>seasonSummary.closedBy</code>	manual-close metadata	yes	nothing

The last two rows are the model's structural weakness. There is no schema version on any seasons document, so a reader cannot tell an archive written by an old build from one written today, and the only defence against silent degradation is two hand-maintained `typeof` checks on two fields.

2. Turning seasons on, off, and on again

Seasons are switched on by one callable, `enableClubSeasons`, and off by another, `disableClubSeasons`. Both live in `functions/src/index.ts` — enable at :16286–16611, disable at :16613–16626. Neither is a document write from the client: `firestore.rules:376–395` makes the `seasons` map server-owned and immutable from the app (`!affectedKeys().hasAny(['seasons']) || request.resource.data.seasons == resource.data.seasons`), and :334–341 forbids a club from being created holding one. Enabling can archive a club's entire history and hand out permanent titles in a single call, so it belongs to the server.

The decision the admin makes is modelled as a **plan** — a pure function, `planActivation`, in `src/utils/seasonActivation.ts:90–176`, with a byte-identical server mirror at `functions/src/seasonActivation.ts` (the only textual difference is the import path, `@/utils/seasonDates` → `./seasonDates`; verified by `diff`). The same function is called three times for one activation: once by the settings screen to decide whether the button is enabled, once implicitly by the confirmation sheet which renders the plan object it is handed, and once by the server, which recomputes it from the club's own data rather than trusting the request.

The inputs, and the branch tree

```
// src/utils/seasonActivation.ts:24–37
export interface ActivationInput {
  cadence: Cadence;           // 'date' | 'rounds'
  months?: number;            // date cadence: season length
  targetRounds?: number;      // rounds cadence: evenings per season
  choice: HistoryChoice;      // 'continue' | 'sealNow'
  playedHistory: number;      // evenings the club has already played
  today: CalendarDate;        // 'YYYY-MM-DD' in the club's timezone
  season1EndsOn?: CalendarDate; // chosen last day of season 1 (date+continue)
  hasHistory?: boolean;       // the club has run seasons before
}
```

`cadence` × `choice` gives four combinations, and `planActivation` treats each separately. **מחזור** (*machzor*, an evening) is the unit throughout — `targetRounds: 24` means twenty-four game-nights, not twenty-four mini-games. Nothing in this file knows that **משחקון** (*mischakon*, a mini-game) exists.

cadence	choice	plan fields returned	meaning
rounds	sealNow	<code>startsAtRounds: 0, roundsRemaining: target</code>	history sealed as season N, season N+1 opens empty
rounds	continue	<code>startsAtRounds: playedHistory, roundsRemaining: target - playedHistory</code>	season N opens already holding the club's history

cadence	choice	plan fields returned	meaning
date	sealNow	<code>startsOn: today , endsOn: seasonEndDate(today, months) , nextStartsOn</code>	history sealed, season N+1 runs <code>months</code> from today
date	continue, date given	<code>endsOn: season1EndsOn , nextStartsOn — no startsOn</code>	season N is transitional: it reaches back as far as the club does
date	continue, no date, <code>hasHistory</code>	<code>endsOn: seasonEndDate(today, months) , nextStartsOn</code>	the re-enable escape hatch, added 18.09
date	continue, no date, first time	error <code>season1EndRequired</code>	the admin must name an end date

The rounds+continue branch carries two refusals that are the heart of the feature's arithmetic (:112–121):

```
if (playedHistory > target!) return { ...base, error: 'historyExceedsTarget' };
if (playedHistory === target) return { ...base, error: 'historyFillsTarget' };
```

A season that opens already at or past its finish line is not a season — it would be archived on the hourly sweep's next pass. `MIN_SEASON_ROUNDS = 2 ( :80 )` is the floor, with no ceiling: "a club that wants a 200-evening season is describing a long season, not a mistake." For the date cadence the bounds are `MIN_SEASON_MONTHS = 1` and `MAX_SEASON_MONTHS = 24 ( src/utils/seasonDates.ts:156–166 )`.

Why the season-1 end date exists, and only there

Under a rounds cadence a transitional first season needs no dates: it starts at `playedHistory` and ends at `target`. Under a **date** cadence with **continue**, there is no honest start date — the history stretches back to the club's first game, months or years before the admin opened the settings screen. The code's answer is to make the first season open-ended at the front and let the admin name its *end*, with the chosen `months` applying only from the following season (`seasonActivation.ts:145–176` ; the server mirrors this at `index.ts:16459–16464` by writing `cadence.startsOn = null , cadence.endsOn = plan.endsOn , cadence.endsAt = null`).

The UI that picks that date is `SeasonsSettings.tsx:818–872` — four chips labelled with the actual dates (`SEASON1_MONTH_CHOICES = [1, 2, 3, 6]` , :67) plus a custom stepper. It is rendered inside `{firstTime ? ... }` (:795), and

```
// src/components/community/SeasonsSettings.tsx:309
const firstTime = (seasons?.count ?? 0) === 0 && !live;
```

Defect `live-club-cannot-use-date-cadence (P0)`. Until commit `ae2c042` (18.09 16:53) `planActivation` returned `season1EndRequired` whenever `season1EndsOn` was missing, with no `hasHistory` escape. Combined with the `firstTime` gate above, the date cadence was *unreachable for the rest of a club's life*: a club already running seasons, or any club with `count > 0` switching the feature back on, saw the enable button permanently disabled and a red line reading "1 בחרו תאריך סיום לעונה 1." ("Choose an end date for season 1.") —

naming a season it had archived, with no date control anywhere on the screen. The server half was patched first (`index.ts:16428–16432` derives the date itself when the client sends none, for the benefit of 1.1.7 clients in the store); the client half followed two commits later.

This fix is not in the shipped app. The store build is 1.1.9 (Android vc235, iOS 103) at commit `4de80d0` , 18.09 05:54Z. `git show 4de80d0:src/utils/seasonActivation.ts | grep -c hasHistory` returns `0`. Every user on the current release still has the dead date cadence; only the server-side derivation protects them, and only when the client lets them press the button, which it does not.

A related residue, ROUND4 finding 21, was fixed at HEAD: the `hasHistory` branch originally returned `ends0n` alone, so the confirmation sheet — which renders each date line only when its field is present (`SeasonConfirmSheet.tsx:180, :186, :196`) — showed an end date and no "מתחילה ב N+1 עונה". `seasonActivation.ts:161–170` now derives `nextStarts0n` too.

Note also that the **server never passes** `hasHistory` (`index.ts:16433–16454`). It reaches the same outcome by a different route, pre-computing `season1Ends0n` itself. Two code paths to one answer is a maintenance hazard in a file whose entire premise is "one plan, computed identically in three places".

The seeding: two counters, two eras

This is the part that produces wrong numbers on real clubs. When a season opens, two fields are stamped onto `groups/{id}.seasons` :

```
// src/utils/seasonSeed.ts:22–30 (and functions/src/seasonSeed.ts, identical)
export function seasonSeed(sealedEvenings: number, playedHistory = 0) {
  return { roundsAtStart: num(sealedEvenings), playedRounds: num(playedHistory) };
}
```

They come from **different sources covering different eras**, and the call site makes that explicit (`index.ts:16573–16576`):

```
...seasonSeed(
  await sealedEveningsOf(groupId),
  closedSoFar > 0 ? 0 : await playedEveningsFromGames(groupId),
),
```

field	source	what it counts	era it can see
<code>roundsAtStart</code>	<code>sealedEveningsOf</code> → <code>clubRecords/{groupId}.eveningsSealed</code> (<code>index.ts:15578–15583</code>)	every evening ever <i>sealed</i> , all-time, never resets, never decrements	from 26.08.2026 only — the counter did not exist before evening-sealing shipped
<code>playedRounds</code>	<code>playedEveningsFromGames</code> → scan of up to 200 terminal <code>games</code> , filtered by <code>didEveningHappen</code> (<code>index.ts:15655–15670</code>)	every evening the club actually held	the club's whole life, capped at 200 documents

`roundsAtStart` is a *zero mark* for the all-time counter, so that every future seal adds exactly one to a season-relative figure. `playedRounds` is the number the club is **shown** on its card and the number the rollover closes on. They are not the same quantity and must not be compared, but because both are "rounds" and both are seeded in the same expression, the codebase has confused them repeatedly.

On the one club that has ever run seasons, `HhzIwmjMl1i5HS0GHt3p` (מועדון שכחת שושי), the gap between the two eras is exactly measurable today:

```
clubRecords/HhzIwmjMl1i5HS0GHt3p.eveningsSealed = 10
games scan (23 terminal docs, 22 with didEveningHappen = 'happened') = 22
```

Twenty-three terminal games run 28.06.2026 → 18.09.2026. Nineteen of them predate the season stamp entirely and carry no `seasonId`; three carry `s1`; one (18.09, `DTNscolR...`) carries `s3` and is `playVerified: false`, so it does not count. The counter sees ten of those twenty-two because it only began on 26.08.

Defect (historic, fixed 16.09 in 5766867, "The season owns the 19 the club played, not the 7 a counter had seen"). `playedRounds` was originally seeded from `eveningsSealed` too. The club turned seasons on, asked for 24, and the card read "7 24 מתוך" ("7 of 24") — not its history (19 at the time), not a fresh start (0), but however far a two-week-old counter had got. The owner's report is quoted verbatim in the source: "19 שיחקנו ולא 7" ("we played 19, not 7"). The fix routes `playedRounds` through the games scan, which is also what the club screen's "מפגשים שנערכו" has always counted.

The client's half of the same number comes from a different function again:

`gameService.getCommunityStats(groupId).totalFinished` (`SeasonsSettings.tsx:340–351`). That scan is also `limit(200)` over `status` in `['finished', 'cancelled']` and also asks `eveningPlayState`, so on the real club the two agree exactly at 22 — but it is a *coincidence of construction*, not a shared module, and `getCommunityStats` additionally applies a season filter (`inSeason`, `src/utils/seasonScope.ts:28–36`) when it is given a scope. The settings screen passes none, so the filter is inert there. Nothing enforces that.

Note the seeding is **skipped for a re-activation**: `closedSoFar > 0 ? 0 : ...`. A club that has closed a season before starts the next one at zero, because those evenings are already sealed inside an archive. Before that guard existed, a club with 22 evenings behind it could not open a 2-round season at all — the server answered `failed-precondition` and the app showed "משהו השתבש" ("something went wrong").

The `sealNow` branch

```
// functions/src/index.ts:16466–16512 (trimmed)
if (choice === 'sealNow') {
  const quiet = await clubIsQuiet(groupId);
  if (!quiet.ok) throw new HttpsError('failed-precondition', quiet.blocker ?? 'busy');
  const firstNo = closedSoFar + 1;
  await closeSeason({
    db, groupId, seasonId: `s${firstNo}`, seasonNo: firstNo,
    startsAt: 0, // display resolves the club's first game
    completedRounds: played, // ← see below
    roundsAtStart: 0,
    partialData: true, // assists only from 21.06, clean sheets from 17.08
    now,
  });
  await ref.set({ seasons: {
    enabled: true, currentNo: firstNo + 1, currentId: `s${firstNo + 1}`,
    startedAt: now, roundsAtStart: sealedAllTime, playedRounds: 0,
    reopenedAt: 0, cadence: rebaseCadence(cadence, now, now),
    targetHistory: [], count: firstNo,
  } }, { merge: true });
  await announceSeasonClosed({ ... });
  return { ok: true, closedSeasonNo: firstNo, currentNo: firstNo + 1 };
}
```

Defect `enable-sealnow-archives-counter-not-games (P1)`. `completedRounds: played` where `played` is computed at `:16328`:

```
const played = await completedRoundsOf(groupId, existing?.roundsAtStart);
```

Two arguments, not three — so `completedRoundsFrom` (`functions/src/seasonCounters.ts:105–118`) takes the fallback path and returns `eveningsSealed - roundsAtStart`. On a first activation `roundsAtStart` is absent, so this is simply `eveningsSealed`. Meanwhile the confirmation sheet the admin approved showed `plan.playedHistory`, which is the **games scan** (`SeasonConfirmSheet.tsx:159`, rendering `he.seasonsConfirmSealedNow(plan.playedHistory)` → "20 20", תיסגר כעת — קיימים מחזורים קיימים existing rounds — will be sealed now"). `closeSeason` writes that argument straight into the archive card (`functions/src/seasonRollover.ts:625–627`). Had שכתה שוישי taken this branch on 16.09, the sheet would have said **20** and the permanent archive would have stored **7**. The archive is created with `summaryRef.create()` — there is no second chance.

The club took `continue` instead, so the defect is unexercised in production; the sealed card `seasonCards/HhzIwmjMl1i5HSOGht3p__s1` today reads `completedRounds: 22`, written later by `endSeasonNow` after its own two-argument bug was repaired. Its `startsAt` is `1789547794676 = 16.09.2026`

08:36Z, the enable moment — which is why the hall of fame captions a season containing three months of football with a day and a half (§11).

Note also `roundsAtStart: sealedAllTime` here versus `seasonSeed(await sealedEveningsOf(...))` in the `continue` branch: the same figure, read twice from the same document in one invocation (`:16338` and `:16574`). Harmless, but it is a second Firestore read per activation.

The `continue` branch and the silent clamp

```
// functions/src/index.ts:16523-16531
if (cadence.type === 'rounds' && closedSoFar === 0) {
  cadence.targetRounds = Math.max(cadence.targetRounds ?? 0, played + 1);
}
```

Defect `rounds-clamp-uses-a-different-history (P2)`. The plan refused a target below `playedHistory` using the **games scan**; this clamp raises the target using `played`, i.e. `eveningsSealed`. Two histories, one decision. When the counter is *lower* than the scan — the normal case today, 10 against 22 — the clamp is a no-op and the plan's refusal governs. When the counter is *higher* — a club past the 200-document scan window, or one that has deleted games (deleting a game decrements nothing, which is precisely why `sealedEveningsOf` exists) — the clamp silently rewrites the admin's choice. The source comment names the failure: an admin choosing 24 gets 201. No `targetHistory` entry is written, no error is returned, and `SeasonConfirmSheet` has already promised 24. This is finding `approved-rounds-target-is-not-the-one-written` from the other side.

Switching off, and switching back on

`disableClubSeasons` is four lines of effect:

```
// functions/src/index.ts:16621-16624
const seasons = group.seasons as { enabled?: boolean } | undefined;
if (!seasons?.enabled) return { ok: true, alreadyOff: true };
await ref.set({ seasons: { enabled: false } }, { merge: true });
```

It deliberately does **not** close the running season — the dialog says so (`he.seasonsDisableBody`): "העונה שרצה" ("the season running now will not be closed and no titles will be awarded for it"). Everything else on the block survives, `count` included, so numbering continues: a club that ran seasons 1–3 re-opens at 4, never at 1.

The same dialog also promises: "ואם תפעילו עונות שוב הספירה תמשיך מהמקום שבו עצרה" — "**and if you turn seasons on again the count will continue from where it stopped.**"

Defect `disable-reenable-zeroes-running-season (P1)`. It does not. Re-enabling runs the `continue` branch with `closedSoFar > 0`, so `seasonSeed(eveningsSealed, 0)` writes `playedRounds: 0` and `startedAt: now`. Every evening the season had accumulated while the feature was on is erased from the card,

and the promise in the dialog is exactly inverted. The one in-between case is a club with `count === 0` that never closed anything: it is re-seeded from the games scan, which is *closer* to correct but still discards whatever the live season held.

QA Test Club (6zotsPlu5Wa18hIysQif) is sitting in that state right now. It is the only other club in the database of 195 groups with a `seasons` block:

```
enabled: false      count: 0      currentNo: 1      currentId: "s1"
playedRounds: 3     roundsAtStart: 0   reopenedAt: 1789509161010 (15.09)
cadence: { type: "rounds", targetRounds: 3, months: null, endsAt: null }
clubRecords/6zotsPlu5Wa18hIysQif: does not exist → sealedEveningsOf = 0
terminal games for this group: 0
```

Press "הפעל עונות" on it today and: `closedSoFar = 0` → `firstTime` on the client is true → `playedHistory = playedEveningsFromGames = 0` → `seasonSeed(0, 0)` → `playedRounds: 0`. Three evenings of progress replaced by zero, and `currentId` re-issued as `s1` — the same id the previous run used. Its `cadence` map also shows the merge hazard the code warns about: it has `months` and `endsAt` keys but no `startsOn` / `endsOn` at all, because it was written by an older build; `merge: true` into a nested map merges field by field, which is why the current enable path writes explicit `null`s on the unused half (`:16362–16368` , `:16386–16394`).

Defect `season-id-reissued-with-live-stamps (P1)`. `count` advances only on a *close*, so disable + enable re-issues the *same* `currentId` while games from the previous run still carry that stamp. The card, seeded to 0, and the season-scoped statistics screen, which counts stamped games via

`playedEveningsOfSeason / countSeasonEvenings` , then answer the same question differently — one says 0, the other says however many games bear the stamp. The live club already shows the general shape of stamp drift: `groups/HhzIwmjMl1i5HS0GHt3p.seasons.currentId` is `"s2"` , and game `DTNscolR...` played on 18.09 is stamped `"s3"` — a season the club document has never heard of.

Defect `reenable-inherits-stale-reopenedat (P2)`. The `sealNow` branch writes `reopenedAt: 0` (`:16502`); the `continue` branch does not write the field at all, and `merge: true` preserves it. `reopenedAt` buys a season 48 hours of immunity from closing (`REOPEN_GRACE_MS` , checked at `:15977–15979` on the seal path and `:16178–16182` in the hourly sweep). So a brand-new season opened by a re-enable within two days of an undo inherits the predecessor's grace and cannot be closed by either mechanism, however far past its target it is. שכיח carries `reopenedAt = 1789727847680 = 18.09.2026 10:37` today.

The same write also sets `targetHistory: []` unconditionally (`:16578`), discarding the audit trail of every finish-line move the club ever made — finding `reenable-wipes-targethistory` . On the live club that array currently holds one entry (a change from `targetRounds: 2` to `targetRounds: 24` by *Eliran Tzabari* at 18.09 10:37:54), and a disable/enable cycle would delete it.

Refusals, and whether the admin ever sees them

Every way `enableClubSeasons` can say no, and what the user gets:

thrown	code	client mapping (seasonService.ts)	Hebrew shown
sign-in required	unauthenticated	none	generic
groupId required	invalid-argument	null unless message contains targetRounds	generic
club not found	not-found	null (only no archive maps)	generic
admin only	permission-denied	notAdmin	"רק מנהל המועדון יכול לסיים עונה."
seasons already on	failed-precondition	seasonsAlreadyOn	"עונות כבר פעילות במועדון. רעננו את המסך..."
targetRounds required	invalid-argument	targetInvalid	"עונה חייבת להכיל לפחות 2 מחזורים."
season-plan: <error> ×6	failed-precondition	lengthInvalid / targetInvalid / historyExceedsTarget / historyFillsTarget / seasonEndRequired / seasonEndPast	the six written explanations
openGame / unsealedGame (sealNow only)	failed-precondition	openGame / unsealedGame	"יש מחזור פתוח במועדון..." / wait-for-seal

The mapping lives in `refusalOf` (`src/services/seasonService.ts:128–183`) and is turned into text by `seasonRefusalText` (`:77–126`), which takes a `SeasonRefusalContext` carrying the real season number — because every one of these messages was originally written about "עונה 1" while numbering continues across a disable/enable cycle.

Defect `season-plan-refusals-unmapped (P2)`. `git show 4de80d0:src/services/seasonService.ts | grep -c 'season-plan:'` returns 0. In the shipped 1.1.9 build all six plan refusals fall through to `he.seasonActionFailed` — "משהו השתבש. נסו שוב עוד רגע" ("Something went wrong. Try again in a moment.") — which is advice that can never work, printed beside a destructive button and an invitation to press it again. The six Hebrew explanations exist in `he.ts` (`:2333–2345`) and are reachable today only when the *client's own* `planActivation` reaches the same verdict first, which by construction it cannot when the disagreement is between client and server histories. The screen also renders the plan error inline (`SeasonsSettings.tsx:901–905`) and disables the button while `!plan.ok` (`:943–949`), so in practice a server-side plan refusal means the two sides disagreed — exactly the case where the generic line is least useful.

Two invariants this area is trying to hold, and their status:

- "The summary a person approves is the thing that actually happens." **Does not hold**. The clamp at `:16531` can raise an approved rounds target, and `sealNow` archives a different evening count from the one the sheet displayed.

- *"Numbering continues across the feature being switched off and on."* **Holds for** `count` **and** `currentNo` , **but not for the data.** The id is reused while old games keep the stamp, and the season's progress is reset to zero against a dialog that promised the opposite.
-